Mawlana Bhashani Science and Technology University
Santosh, Tangail–1902, Bangladesh
Department of Information and Communication Technology

PROJECT PROPOSAL

Course Code: ICT-3108
Course Title: Software Engineering and Project Management

CyberSecurity Monitor

A Java-Based Desktop Application for Login Monitoring, Security Checklist Auditing and Software Update Tracking

Submitted by

Name: Alif Ahmed Joy
ID: IT24015
3rd Year 1st Semester
Session: 2023–24
Department of ICT, MBSTU

Supervised by

Dr. Ziaur Rahman
Professor
Dept. of ICT, MBSTU

Date of Submission: October 2026

Abstract

Individuals and small organisations increasingly depend on computers for study, work and communication, yet most of them have no simple way to see whether their systems are being protected properly. Repeated failed logins, missing security practices and outdated software are three of the most common weaknesses that go unnoticed until an incident occurs.

This proposal presents **CyberSecurity Monitor**, a lightweight Java desktop application that brings these three concerns into one place. The system provides (1) a *security checklist* of recommended practices with a saved completion score, (2) a *login monitor* that records login events and raises a security alert after repeated failed attempts, (3) a *software update tracker*, and (4) a *summary report generator* that reads all stored records and rates the overall risk level. All data are stored in plain text files, so the application needs no database server and can run on any computer with a Java Runtime Environment.

The project follows an **incremental, Scrum-based (agile) process model** and is planned over 16 weeks in four development sprints. This document contains the complete proposal: problem definition, objectives, feasibility, requirements, process model, schedule with Gantt chart, milestones and deliverables, budget, risk and quality plans, the UML models (data flow, use case, activity, entity–relationship, class and object diagrams), the workflow generated from Mermaid, the user-interface design and a working skeleton of the Java source code with at least five classes.

Keywords: cybersecurity, login monitoring, security checklist, software updates, Java Swing, UML, Scrum.

Contents

Abstract i

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Motivation . . . 1
- 1.4 Proposed Solution in Brief . . . 1

2 Objectives and Scope 2
- 2.1 General Objective . . . 2
- 2.2 Specific Objectives . . . 2
- 2.3 Scope . . . 2
- 2.4 Expected Users . . . 2

3 Literature Review and Existing Systems 3

4 Feasibility Study 4

5 Requirement Specification 5
- 5.1 Functional Requirements . . . 5
- 5.2 Non-Functional Requirements . . . 5
- 5.3 Hardware and Software Requirements . . . 6

6 Software Process Model 7
- 6.1 Selected Model: Incremental Development with Scrum . . . 7
  - 6.1.1 Why this model? . . . 7
- 6.2 Phases and Sprints . . . 8

7 Project Plan 9
- 7.1 Work Breakdown Structure . . . 9
- 7.2 Project Completion Schedule . . . 9
- 7.3 Gantt Chart . . . 10
- 7.4 Milestones . . . 11
- 7.5 Deliverables . . . 11
- 7.6 Resources and Budget . . . 12
- 7.7 Risk Management . . . 12
- 7.8 Quality Assurance and Configuration Management . . . 12

7.9 Communication Plan 13

8 System Analysis and Design **14**
8.1 Project Work Flow 14
8.2 Data Flow Diagrams (DFD) 15
8.2.1 Level 0 – Context Diagram 15
8.2.2 Level 1 DFD 16
8.3 Use Case Diagram 17
8.4 Activity Diagrams 18
8.4.1 Overall application activity 18
8.4.2 Activity diagram with swim-lanes – Record Login Event 19
8.5 Entity–Relationship Diagram (ERD) 19
8.6 Class Diagram 21
8.7 Object Diagram 22

9 Implementation Plan and User Interface **23**
9.1 Project Structure 23
9.2 Class Responsibilities 23
9.3 Report Risk Rule 24
9.4 User Interface Design 24
9.5 Build and Run Instructions 25
9.6 Source Code of the Initial Version 25

10 Testing Plan **33**

11 Expected Outcomes, Limitations and Future Work **34**
11.1 Expected Outcomes 34
11.2 Limitations 34
11.3 Future Work 34
11.4 Conclusion 34

References **35**

A Sample Data Files **36**

List of Figures

6.1 Scrum-based incremental cycle used in this project 7

7.1 Gantt chart of the 16-week project schedule (bars = tasks, diamonds = milestones) 10

8.1 Project work flow (generated from Mermaid code) 14
8.2 DFD Level 0 (context diagram) 15
8.3 DFD Level 1 – main processes and data stores 16
8.4 Use case diagram 17
8.5 Activity diagram of the whole application 18
8.6 Activity diagram with swim-lanes for the login-monitoring process . . . 19
8.7 Entity–relationship diagram 20
8.8 Class diagram (six project classes plus the nested `LoginEvent` class) . . 21
8.9 Object diagram – snapshot of the system with sample data 22

9.1 Interface design (the implemented Swing windows follow this layout) . 24

List of Tables

3.1 Comparison with existing solutions 3

4.1 Feasibility analysis 4

5.3 Development and runtime environment 6

6.1 Phases, sprints and the increment produced 8

7.1 Work breakdown structure (WBS) 9
7.2 Detailed project schedule 9
7.3 Project milestones 11
7.4 Project deliverables 11
7.5 Estimated budget (Bangladeshi Taka) 12
7.6 Risk register (P = probability, I = impact; L/M/H) 12

8.1 Mapping of the work flow to classes and methods 15
8.2 Use case description: UC2 – Record Login Event 18
8.3 Mapping of entities to text files 20

9.1 Classes used in the implementation (all appear in the class diagram) . . 23

10.1 Planned test cases 33

1 Introduction

1.1 Background

Cybersecurity is no longer a concern of large enterprises alone. Students, teachers, small offices and home users store personal data, research files and financial information on their computers. Attackers exploit very basic weaknesses: weak or reused passwords, brute-force login attempts, missing security patches and the absence of simple protective habits. Frameworks such as the NIST Cybersecurity Framework [7] and the CIS Critical Security Controls [8] stress that *monitoring*, *access control* and *timely patching* are among the most effective low-cost defences.

Professional monitoring platforms (SIEM systems) are powerful but expensive, complex to install and difficult for beginners to understand. There is therefore a gap for a small, easy-to-use, educational tool that demonstrates the core ideas of security monitoring while remaining practical for everyday use.

1.2 Problem Statement

Users currently lack a single, simple application that

- tells them which basic security practices they have or have not completed,
- records login events and warns them when repeated failed logins occur,
- keeps track of which installed software still needs an update, and
- summarises all of this in one understandable report.

As a result, security status is checked manually and irregularly, early signs of an attack are missed, and outdated software remains unpatched.

1.3 Motivation

- Repeated failed logins are a classic indicator of password-guessing attacks.
- Unpatched software is a leading cause of successful intrusions.
- The project lets the author apply object-oriented programming, file handling, GUI development and software-engineering practice (process model, UML, testing) in one realistic system.

1.4 Proposed Solution in Brief

CyberSecurity Monitor is a Java Swing desktop application with a main menu of six options (Figure 8.1): *Security Checklist*, *Record Login Event*, *View Login Events*, *Track Software Updates*, *Generate Security Report* and *Exit*. Records are persisted in three text files (`security.txt`, `login_events.txt`, `software_updates.txt`) managed by a dedicated `FileManager` class.

2 Objectives and Scope

2.1 General Objective

To design, plan and develop a simple, reliable, file-based Java application that helps users monitor and improve the basic security posture of their computer.

2.2 Specific Objectives

1. To provide an interactive security checklist and compute a security score.
2. To record every login event (success or failure) with a time stamp and user name.
3. To detect *repeated failed logins* (three or more consecutive failures for the same user) and display a security alert.
4. To track the update status (up-to-date or pending) of installed software.
5. To generate a summary report that combines all stored data and estimates a risk level (LOW, MEDIUM, HIGH).
6. To follow a documented software process, produce UML models and deliver a tested system in 16 weeks.

2.3 Scope

In scope	**Out of scope**
Desktop GUI application in Java (Swing)	Web or mobile versions
Manual/semi-automatic entry of login events	Automatic capture of operating-system logs
Text-file storage in the `data/` folder	Database server, cloud storage
Alert after repeated failed logins	E-mail/SMS notification, active blocking of accounts
Update status recorded by the user	Automatic downloading or installing of updates
Summary report on screen	Encrypted storage, multi-user authentication, PDF export

2.4 Expected Users

Students and teachers, home users, and small-office administrators who want a quick, readable overview of their security status without installing heavy enterprise tools.

3 Literature Review and Existing Systems

Several tools already address parts of the problem. Table 3.1 compares them with the proposed system.

Table 3.1: Comparison with existing solutions

Tool	Strength	Limitation for beginners	CyberSecurity Monitor
Operating-system event viewer	Detailed, authoritative log of logins	Hard to read; no alerts or summary	Simple table and automatic alert
Open-source host monitors (e.g. Wazuh, OSSEC)	Full intrusion detection	Server setup and configuration required	Zero setup, single JAR
Commercial SIEM (e.g. Splunk)	Enterprise analytics	Costly; steep learning curve	Free, educational
Antivirus suites	Malware protection	No checklist, no login tracking	Combines checklist, logins, updates

The proposed application does not compete with these products. Its contribution is *integration and simplicity*: three of the most important everyday security indicators – good practices, authentication events and patch status – in one small program that is easy to understand, extend and teach. The design follows the guidance of the NIST framework [7], the CIS controls [8] and the OWASP authentication recommendations [9], which advise monitoring and limiting repeated failed authentication attempts.

4 Feasibility Study

Table 4.1: Feasibility analysis

Type	Assessment
Technical	Java and Swing are mature, free and well documented. File I/O uses the standard `java.nio.file` API. The author has prior knowledge of Java and OOP. **Feasible.**
Economic	Only free tools are required (OpenJDK, a free IDE, LaTeX). Estimated cost is about BDT 9,000 (Table 7.5). **Feasible.**
Operational	Simple menu-driven GUI; no training needed; runs on Windows, Linux and macOS. **Feasible.**
Schedule	Four short sprints with a working increment at the end of each. 16 weeks is sufficient for one developer. **Feasible.**
Legal/Ethical	The tool stores only user names and event types entered by the user; no passwords are stored. **Acceptable.**

5 Requirement Specification

5.1 Functional Requirements

ID	Requirement	Priority
FR-1	The system shall display a main menu with six numbered options after launch.	High
FR-2	The system shall show the security checklist and let the user tick completed items.	High
FR-3	The system shall save checklist status to `security.txt` and compute a score.	High
FR-4	The system shall record a login event (user name, event type, time stamp) in `login_events.txt`.	High
FR-5	The system shall reject an empty user name.	Medium
FR-6	The system shall display a security alert when a user has three or more consecutive failed logins.	High
FR-7	The system shall display all saved login events in a table.	High
FR-8	The system shall record software name, version and update status in `software_updates.txt`.	Medium
FR-9	The system shall generate a summary report (checklist score, login statistics, pending updates, risk level).	High
FR-10	The system shall return to the main menu after each task and ask for confirmation before exit.	Medium
FR-11	The system shall create the `data/` folder and files automatically if they do not exist.	Medium

5.2 Non-Functional Requirements

ID	Category	Requirement
NFR-1	Usability	A new user shall be able to complete any task within three clicks from the main menu.
NFR-2	Performance	Any menu action shall respond in under one second for up to 10,000 stored records.
NFR-3	Reliability	Corrupted or incomplete lines in data files shall be ignored without crashing.
NFR-4	Portability	The program shall run on any platform with Java 11 or later.

ID	Category	Requirement
NFR-5	Maintainability	Each concern shall be in its own class; public methods documented with Javadoc comments.
NFR-6	Security	No passwords shall be stored; data files stay on the local computer.

5.3 Hardware and Software Requirements

Table 5.3: Development and runtime environment

Hardware	PC/laptop, 4 GB RAM, 500 MB free disk, 1280×720 display
Operating system	Windows 10/11, Ubuntu Linux or macOS
Language / GUI	Java (JDK 17), Swing
IDE	NetBeans, IntelliJ IDEA Community or VS Code
Storage	Plain text files in `data/`
Version control	Git (local repository, optionally GitHub)
Documentation	LaTeX (Overleaf or TeX Live), draw.io/Mermaid for diagrams
Testing	Manual test cases (Chapter 10); optional JUnit 5

6 Software Process Model

6.1 Selected Model: Incremental Development with Scrum

The project uses the **incremental process model organised as Scrum sprints** [1, 3]. The system is naturally divided into independent modules (file management, login monitoring, checklist, update tracking, reporting). Each module is planned, designed, coded, tested and demonstrated as a working *increment* at the end of a two-week sprint.

6.1.1 Why this model?

- **Early working software:** a usable menu and file layer exist after Sprint 1.
- **Low risk:** each increment is tested on its own before integration.
- **Flexible:** the supervisor's feedback after each sprint review can change the backlog (e.g. threshold for failed logins).
- **Suited to one developer:** Scrum is scaled down – the student is the Developer/Scrum Master, the supervisor acts as Product Owner.

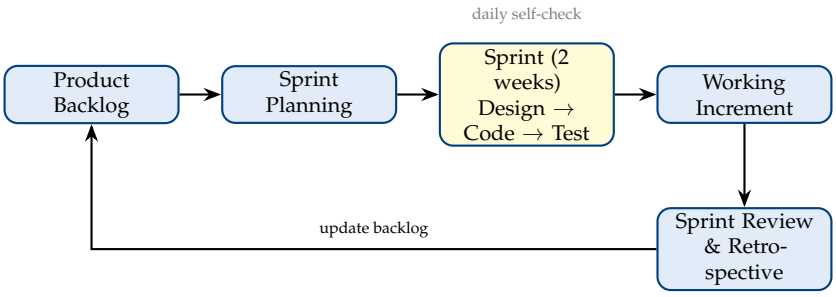


Figure 6.1: Scrum-based incremental cycle used in this project

6.2 Phases and Sprints

Table 6.1: Phases, sprints and the increment produced

Phase / Sprint	Weeks	Goal and increment
Initiation	1–2	Proposal, objectives, feasibility, project plan
Requirements	2–3	Requirement specification (SRS), use cases, product backlog
Design	3–4	DFD, use case, activity, ER, class and object diagrams; UI mock-ups
Sprint 1	5–6	`FileManager`, `Main` window and menu → *v0.1*
Sprint 2	7–8	`LoginMonitor`: record/view events, repeated-failure alert → *v0.2*
Sprint 3	9–10	`SecurityChecklist` and `SoftwareUpdateTracker` → *v0.3*
Sprint 4	11–12	`SecurityReport`, integration, UI polish → *v0.4 (release candidate)*
Testing	13–14	System testing, bug fixing, performance check → *v1.0*
Closure	12–16	Final report, user manual, presentation, submission

7 Project Plan

7.1 Work Breakdown Structure

Table 7.1: Work breakdown structure (WBS)

ID	Work package	Activities
1.0	Project management	Proposal, plan, supervisor meetings, risk log, status reports
2.0	Requirements	Interview/read sources, write FR/NFR, use cases, backlog
3.0	Design	UML models, file formats, UI mock-ups, class design
4.0	Implementation	4.1 FileManager & Main; 4.2 LoginMonitor; 4.3 Checklist & Updates; 4.4 Report
5.0	Testing	Test plan, unit tests, integration tests, system test, bug fixing
6.0	Documentation	Code comments, user manual, final report, slides
7.0	Delivery	Demo, supervisor review, final submission

7.2 Project Completion Schedule

Project start: **Monday, 2 November 2026**; planned completion: **Friday, 19 February 2027** (16 weeks, about 10 working hours per week, ≈160 hours).

Table 7.2: Detailed project schedule

ID	Task	Weeks	Dates (2026–27)
T1	Project initiation and proposal	1–2	2 Nov – 13 Nov
T2	Requirement analysis (SRS)	2–3	9 Nov – 20 Nov
T3	System design and UML modelling	3–4	16 Nov – 27 Nov
T4	Sprint 1 – core framework	5–6	30 Nov – 11 Dec
T5	Sprint 2 – login monitor	7–8	14 Dec – 25 Dec
T6	Sprint 3 – checklist and updates	9–10	28 Dec – 8 Jan
T7	Sprint 4 – report and integration	11–12	11 Jan – 22 Jan
T8	System testing and bug fixing	13–14	25 Jan – 5 Feb
T9	Documentation and user manual	12–15	18 Jan – 12 Feb
T10	Final presentation and submission	15–16	8 Feb – 19 Feb

7.3 Gantt Chart

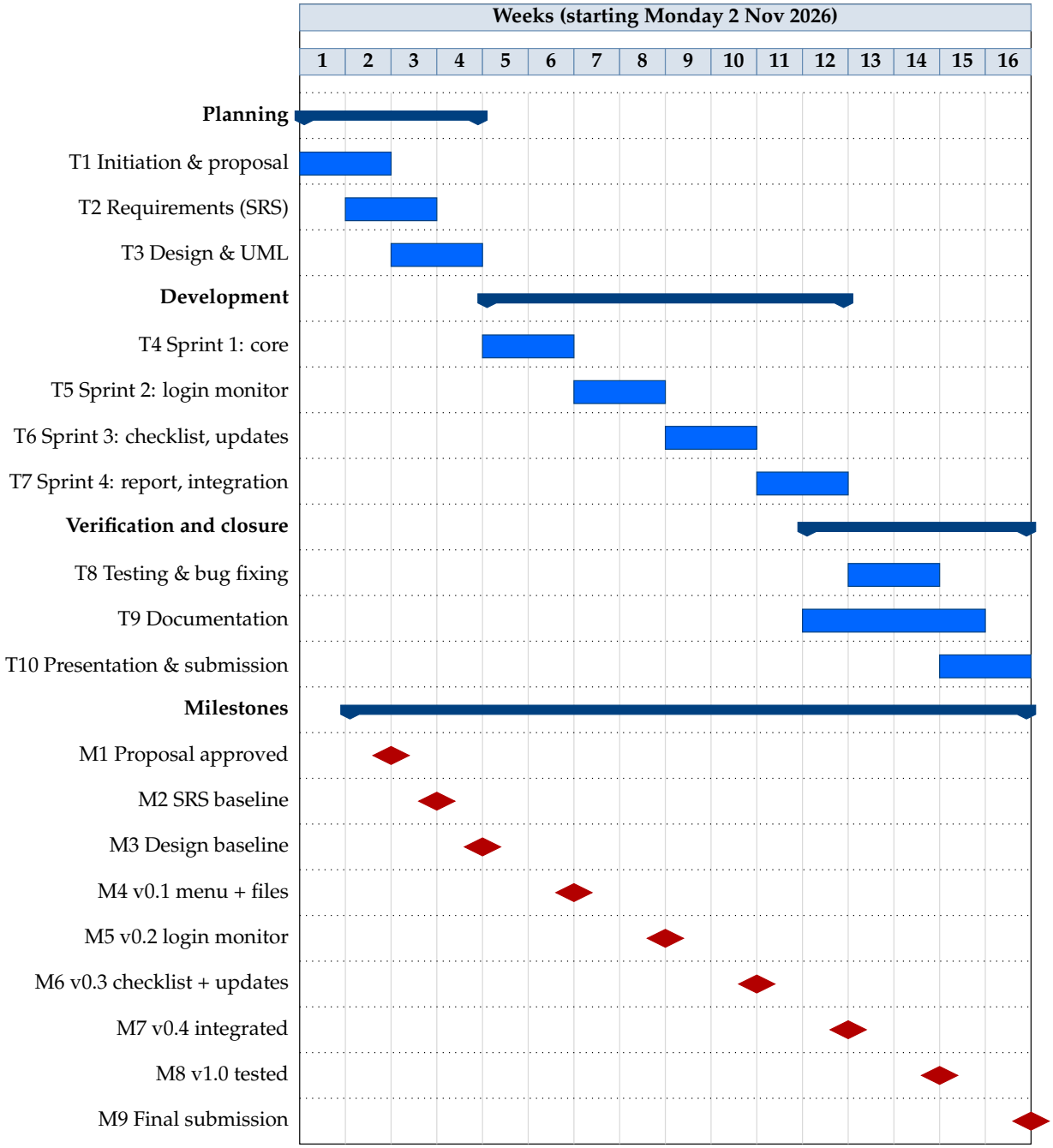


Figure 7.1: Gantt chart of the 16-week project schedule (bars = tasks, diamonds = milestones)

7.4 Milestones

Table 7.3: Project milestones

ID	Milestone	Week	Date	Completion criterion
M1	Proposal approved	2	13 Nov 2026	Supervisor signs off proposal
M2	SRS baseline	3	20 Nov 2026	Requirements frozen
M3	Design baseline	4	27 Nov 2026	All six UML diagrams complete
M4	Prototype v0.1	6	11 Dec 2026	Menu opens; files auto-created
M5	Login module v0.2	8	25 Dec 2026	Events saved; alert works
M6	Modules v0.3	10	8 Jan 2027	Checklist and update tracker work
M7	Integrated v0.4	12	22 Jan 2027	Report generated; all menu options work
M8	Tested release v1.0	14	5 Feb 2027	All test cases pass
M9	Final submission	16	19 Feb 2027	Report, code, slides delivered

7.5 Deliverables

Table 7.4: Project deliverables

ID	Deliverable	Due	Format
D1	Project proposal (this document)	Wk 2	LaTeX source + PDF
D2	Software Requirement Specification	Wk 3	PDF
D3	UML design document (DFD, use case, activity, ERD, class, object)	Wk 4	PDF/PNG
D4	Prototype v0.1 (menu + file layer)	Wk 6	Source code (`.java`)
D5	Login monitor module v0.2	Wk 8	Source code + test log
D6	Checklist and update tracker v0.3	Wk 10	Source code + test log
D7	Integrated application v0.4	Wk 12	Executable JAR
D8	Test plan and test report	Wk 14	PDF
D9	User manual	Wk 15	PDF
D10	Final project report, source code, slides	Wk 16	PDF, ZIP, PPTX

7.6 Resources and Budget

Table 7.5: Estimated budget (Bangladeshi Taka)

Item	Cost (BDT)
Development computer (already owned)	0
OpenJDK, IDE, Git, LaTeX (free software)	0
Internet and data	4,000
Printing and binding of reports	2,500
Reference books and learning material	1,500
Contingency	1,000
Total	**9,000**

7.7 Risk Management

Table 7.6: Risk register (P = probability, I = impact; L/M/H)

ID	Risk	P	I	Mitigation
R1	Scope creep (extra features)	M	M	Freeze backlog at M2; new ideas go to "future work"
R2	Lack of time because of exams/lab work	H	M	Buffer in weeks 14–16; work on small tasks daily
R3	Data file corruption or loss	M	H	Ignore bad lines; keep backups; Git commits
R4	Misunderstanding of requirements	L	H	Review with supervisor at every sprint review
R5	Bugs found late	M	M	Test each increment immediately; test plan in advance
R6	Computer failure	L	H	Remote Git repository and weekly copy to cloud/USB

7.8 Quality Assurance and Configuration Management

- **Standards:** Java naming conventions, Javadoc comments, one class per file.
- **Reviews:** self-review checklist before each commit, supervisor review at each sprint end.
- **Testing:** unit tests for `LoginMonitor` and `SoftwareUpdateTracker`; manual GUI tests (Chapter 10).
- **Version control:** Git with a commit per feature and a tag per milestone (v0.1 … v1.0).

- **Change control:** changes accepted only at sprint planning and recorded in the backlog.

7.9 Communication Plan

Weekly short meeting or message update with the supervisor; sprint review demonstration every two weeks; documents shared through a cloud folder; final presentation in week 16.

8 System Analysis and Design

8.1 Project Work Flow

The work flow of the application was generated from Mermaid code and is used as the master process of this project. Figure 8.1 shows how the program starts, displays the main menu, branches on the option selected (1–6) and returns to the menu after every task. The only exit is option 6. The decision "Repeated Failed Logins?" in the login branch leads either to *Display Security Alert* (yes) or *Continue* (no).

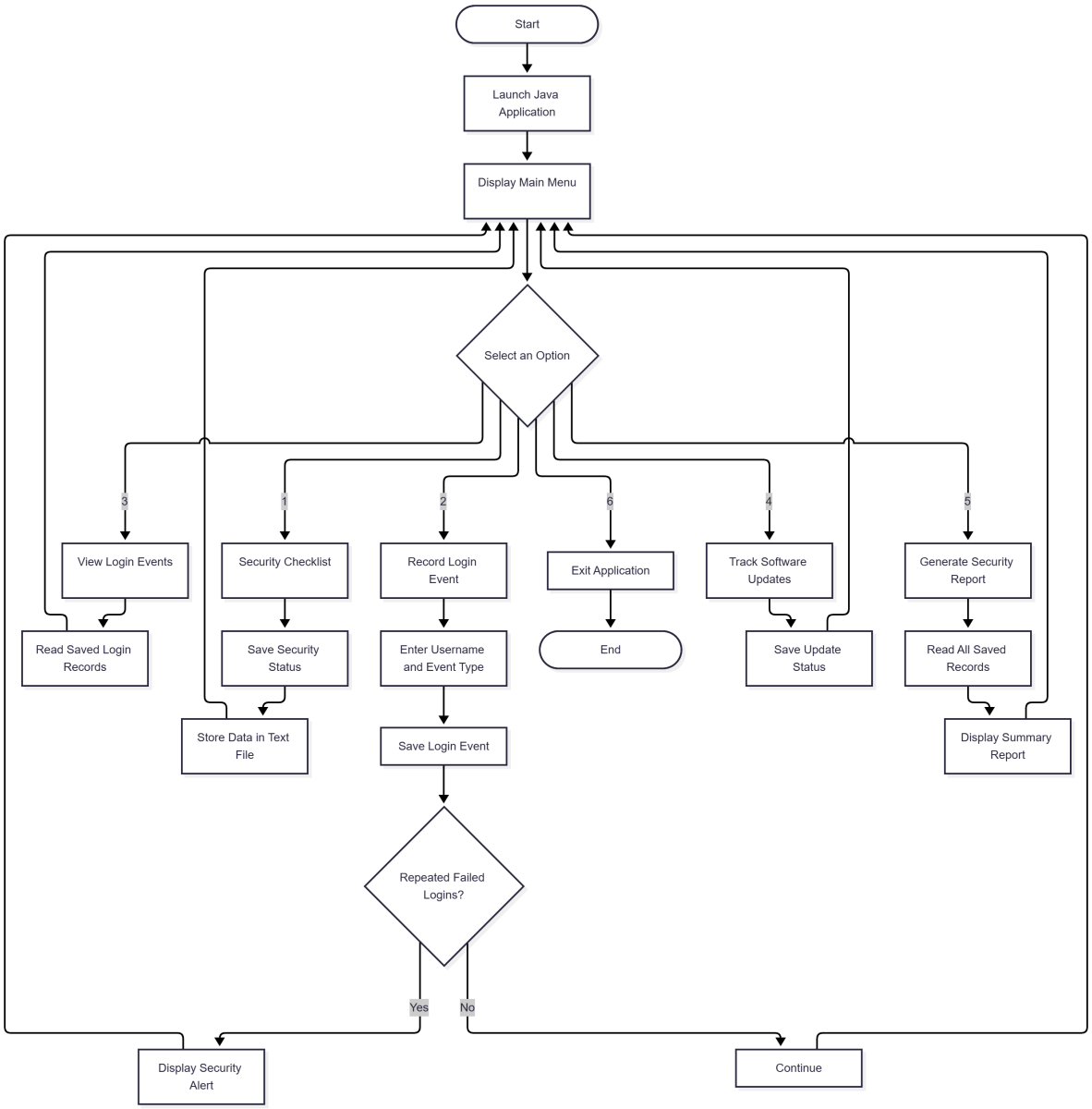


Figure 8.1: Project work flow (generated from Mermaid code)

Table 8.1: Mapping of the work flow to classes and methods

Menu option	Responsible class	Workflow steps → method
Start / Launch / Main Menu	`Main`	`main()`, constructor, `buildMenu()`
1 Security Checklist	`SecurityChecklist`	Save Security Status → `save()`; Store Data in Text File → `FileManager.writeAll()`
2 Record Login Event	`LoginMonitor`	Enter Username and Event Type → `Main.recordLogin()`; Save Login Event → `record()`; Repeated Failed Logins? → `hasRepeatedFailures()`
3 View Login Events	`LoginMonitor`	Read Saved Login Records → `getEvents()`
4 Track Software Updates	`SoftwareUpdateTracker`	Save Update Status → `saveUpdate()`
5 Generate Security Report	`SecurityReport`	Read All Saved Records, Display Summary Report → `generate()`
6 Exit Application	`Main`	`exitApplication()`

8.2 Data Flow Diagrams (DFD)

8.2.1 Level 0 – Context Diagram

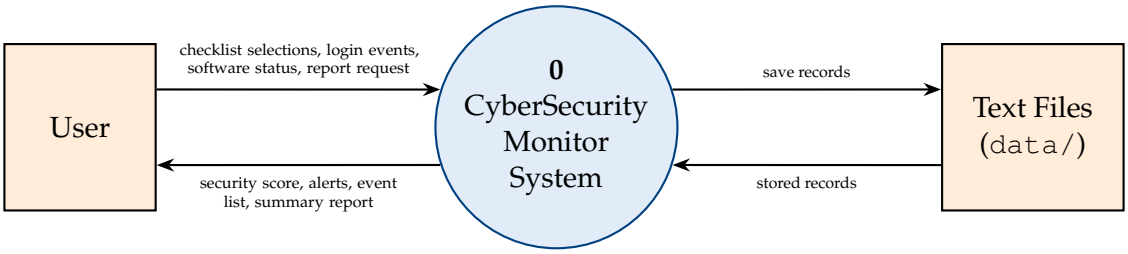


Figure 8.2: DFD Level 0 (context diagram)

8.2.2 Level 1 DFD

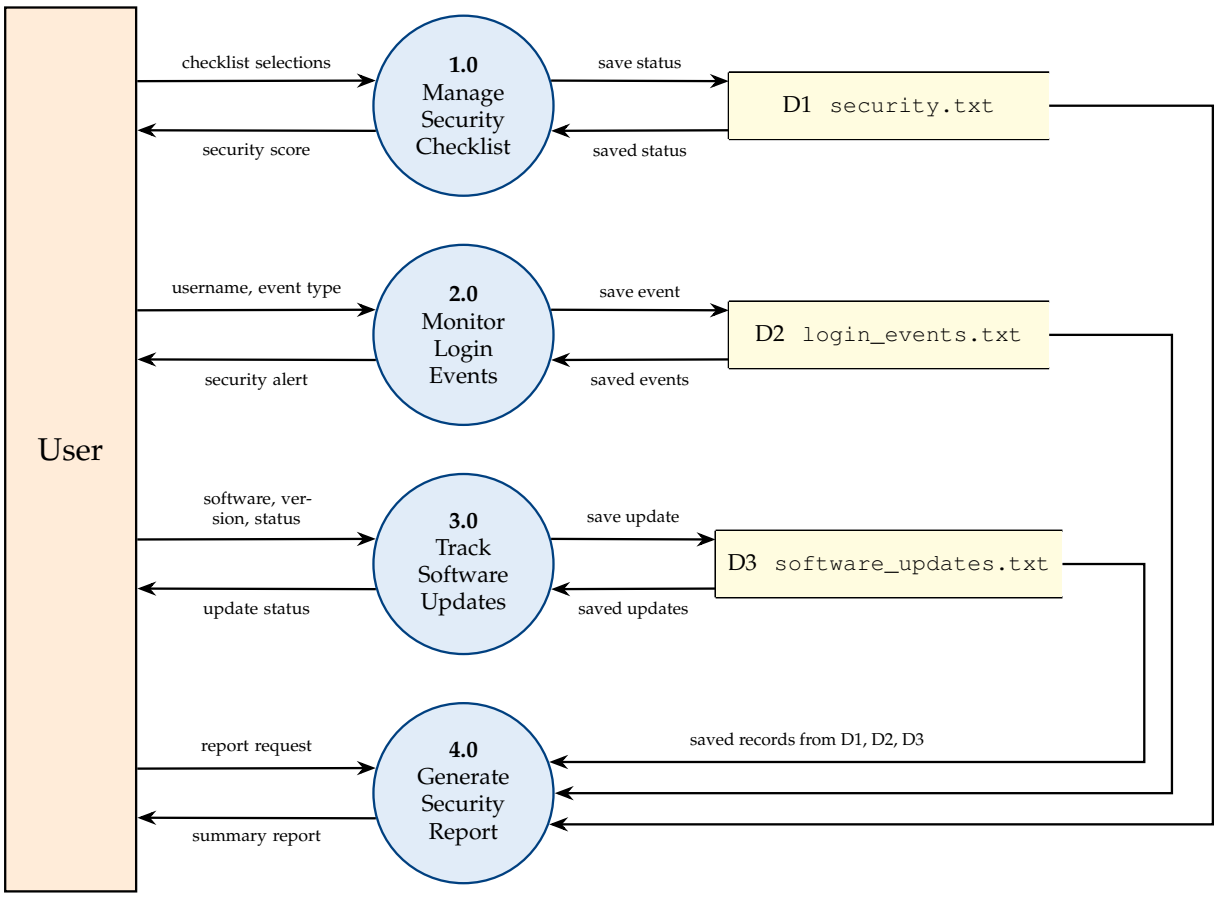


Figure 8.3: DFD Level 1 – main processes and data stores

8.3 Use Case Diagram

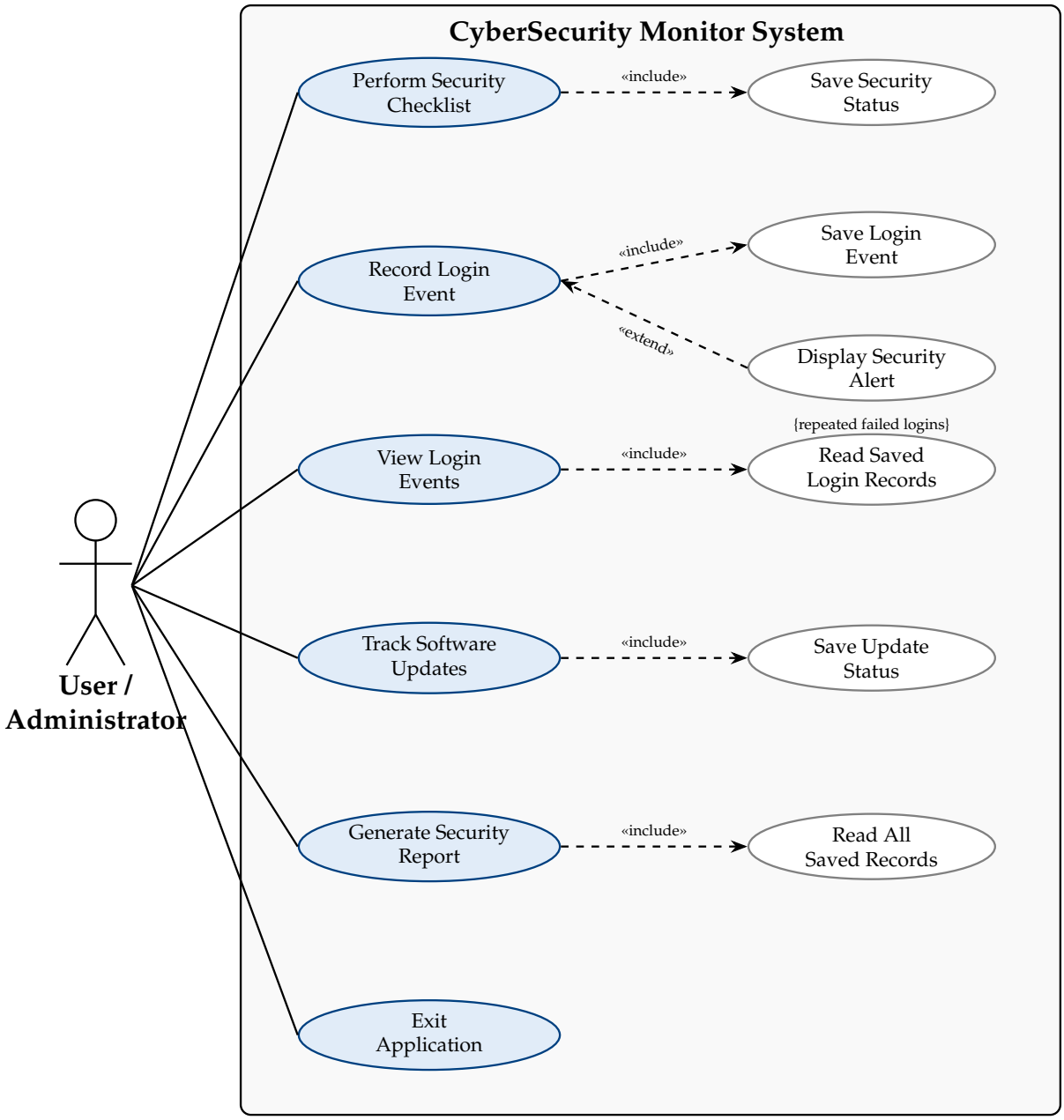


Figure 8.4: Use case diagram

Table 8.2: Use case description: UC2 – Record Login Event

Actor	User / Administrator
Pre-condition	Application is running and the main menu is displayed
Main flow	1) User selects option 2. 2) System asks for user name and event type. 3) User confirms. 4) System saves the event with a time stamp. 5) System checks for repeated failed logins. 6) System returns to the main menu.
Alternative flow	3a) User name is empty → error message, no record saved.
Extension	5a) Three or more consecutive failures → *Display Security Alert*.
Post-condition	Event is stored in `login_events.txt`

8.4 Activity Diagrams

8.4.1 Overall application activity

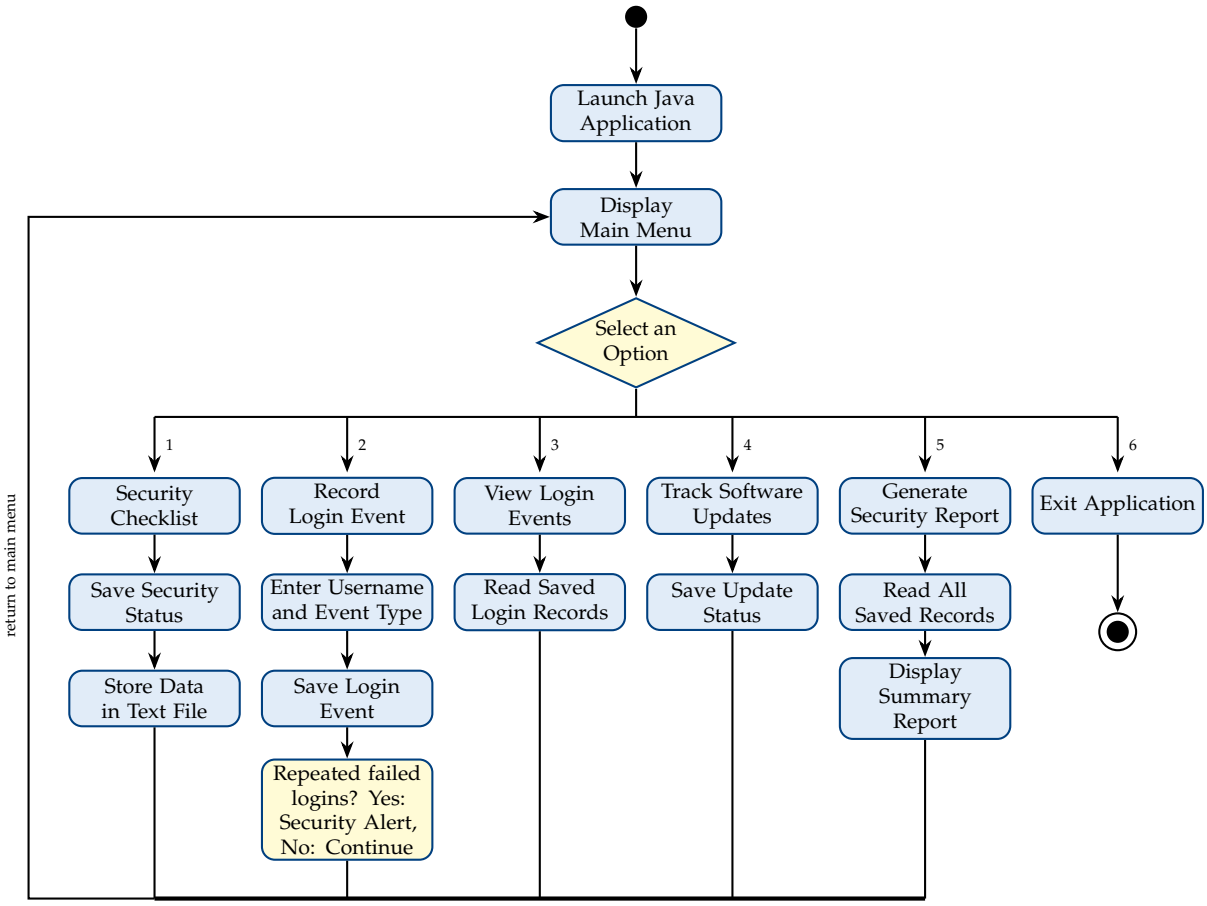


Figure 8.5: Activity diagram of the whole application

8.4.2 Activity diagram with swim-lanes – Record Login Event

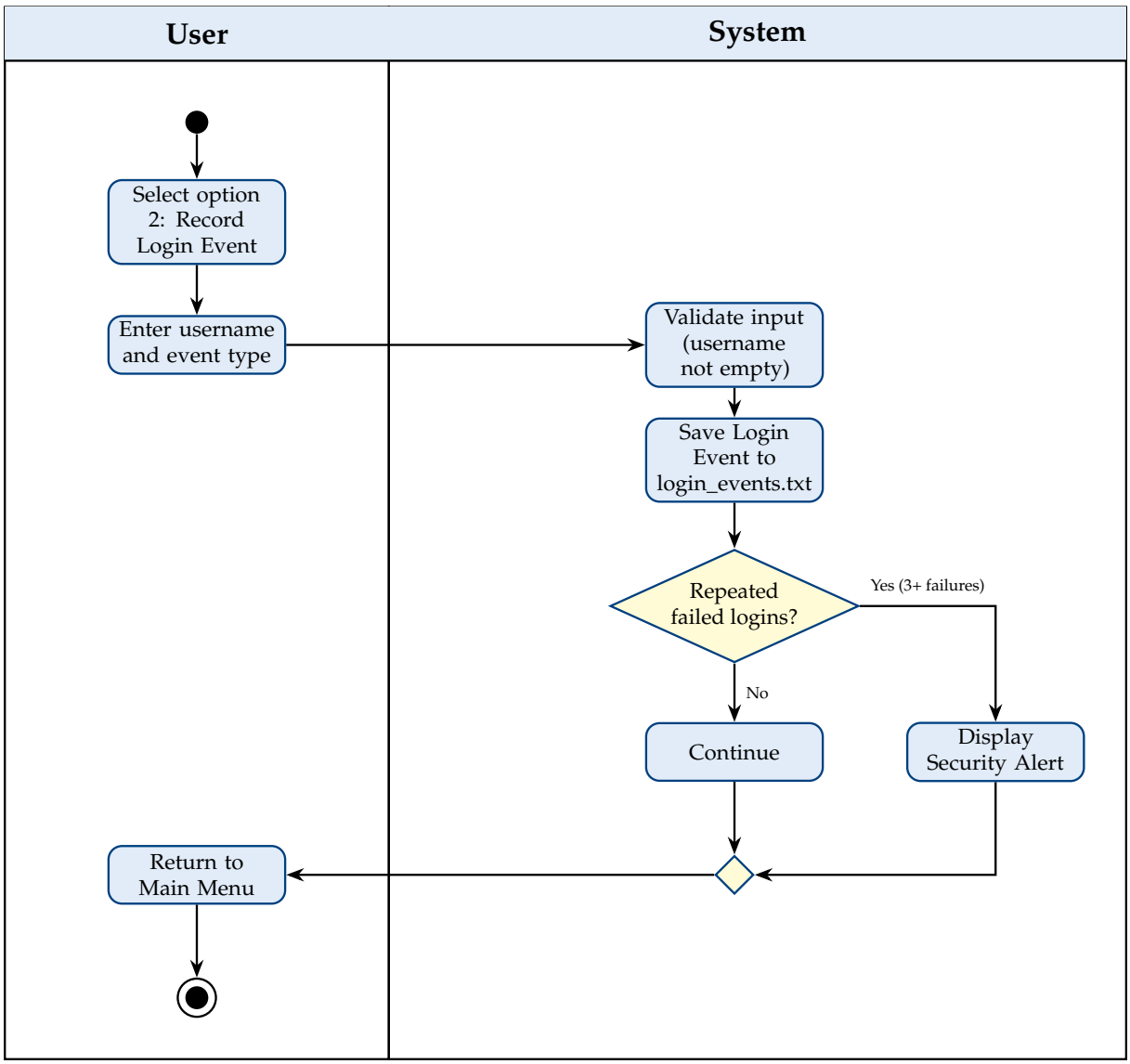


Figure 8.6: Activity diagram with swim-lanes for the login-monitoring process

8.5 Entity–Relationship Diagram (ERD)

Although data are stored in text files, the logical data model is expressed as an ERD so that the files could later be migrated to a relational database without redesign.

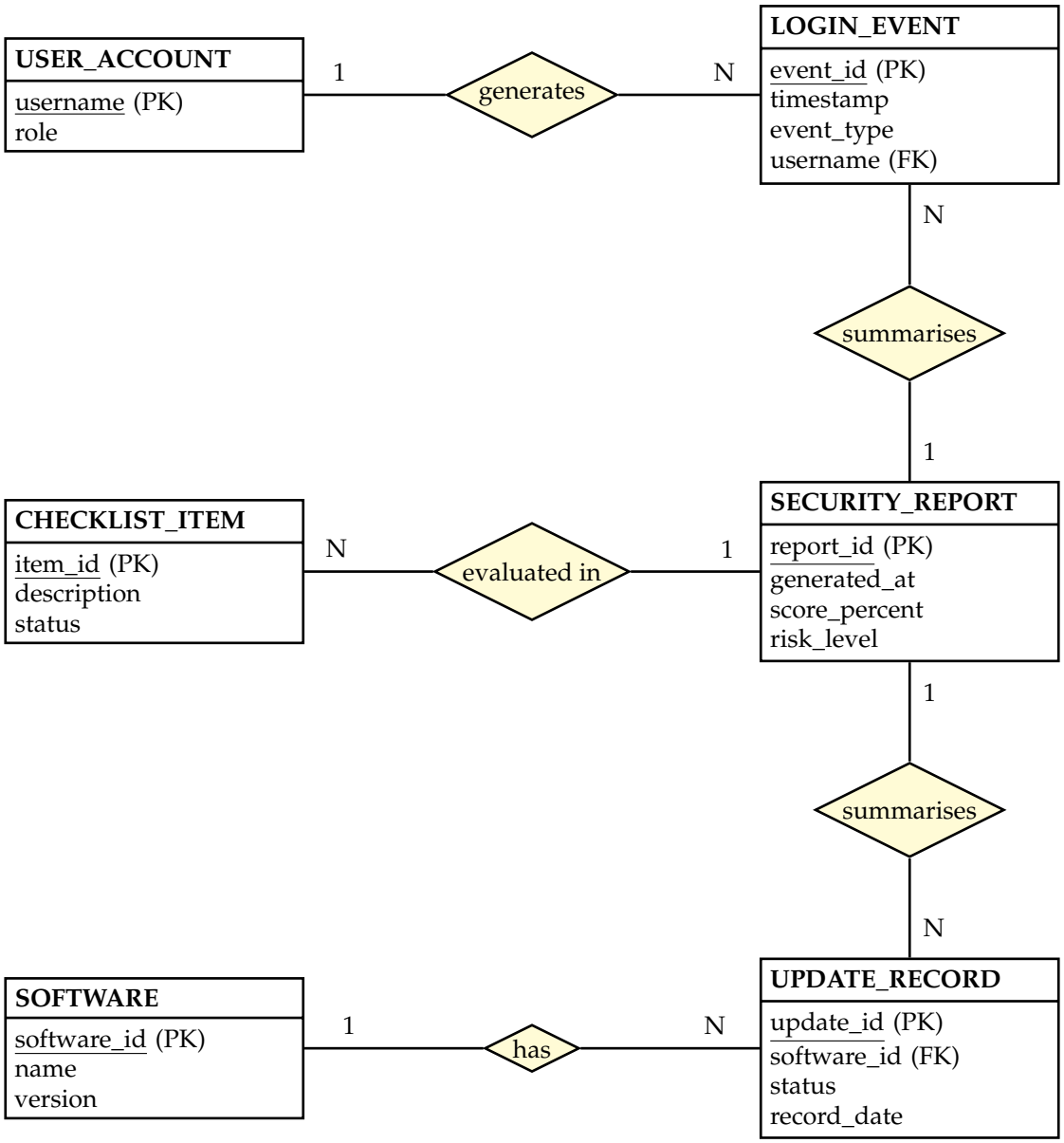


Figure 8.7: Entity–relationship diagram

Table 8.3: Mapping of entities to text files

File	Entity	Record format (one line = one record)
`login_events.txt`	LOGIN_EVENT	`timestamp|username|eventType`
`security.txt`	CHECKLIST_ITEM	`description|DONE or PENDING`
`software_updates.txt`	SOFTWARE, UPDATE_RECORD	`software|version|status|date`
`(computed at run time)`	SECURITY_REPORT	Generated by `SecurityReport.generate()`

8.6 Class Diagram

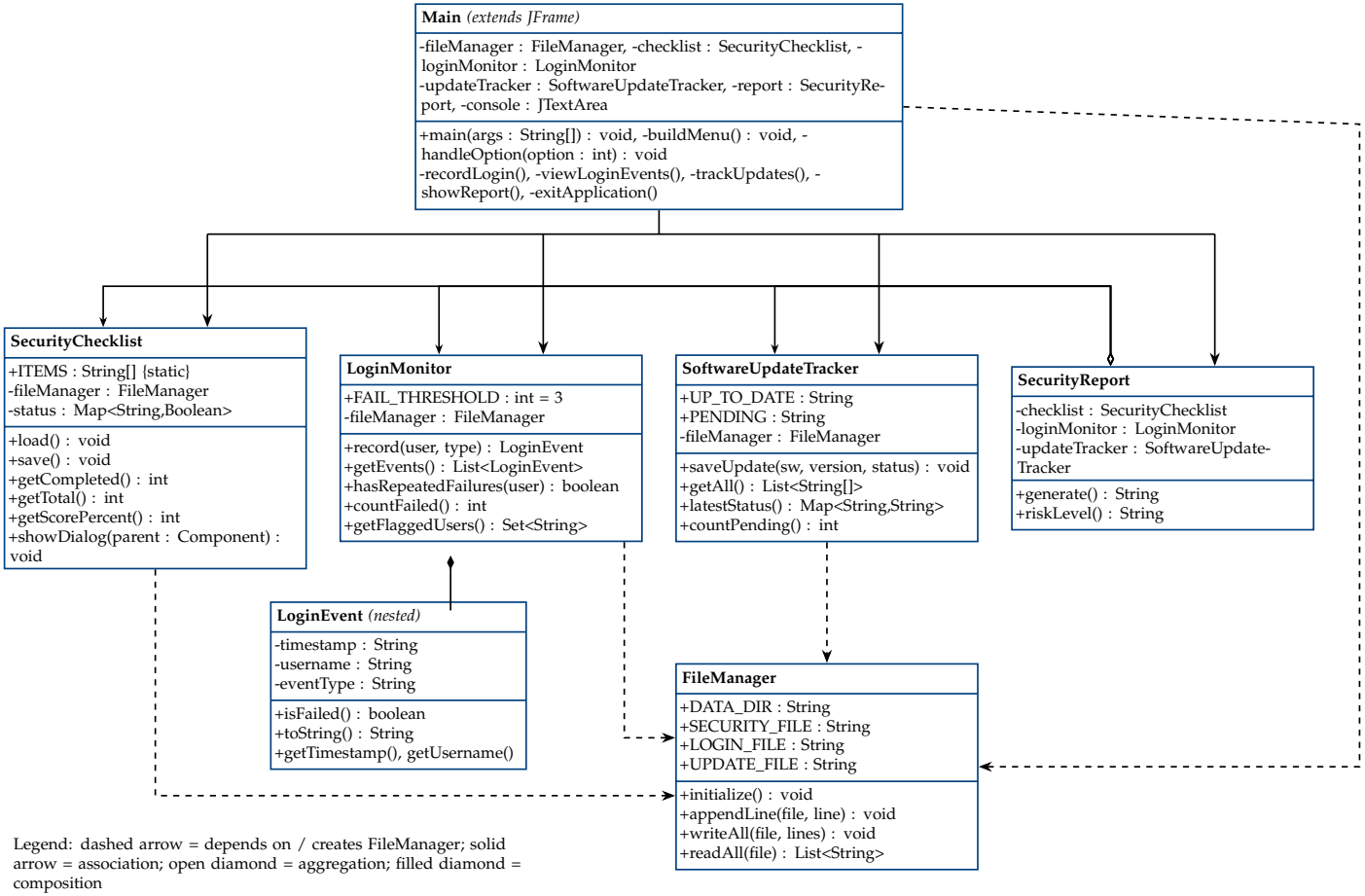


Figure 8.8: Class diagram (six project classes plus the nested `LoginEvent` class)

8.7 Object Diagram

The object diagram shows a snapshot of the running system using the sample data of Appendix A: seven login events (four failed, with user *rahim* failing three times in a row), eight checklist items of which five are done, and four tracked programs with two pending updates.

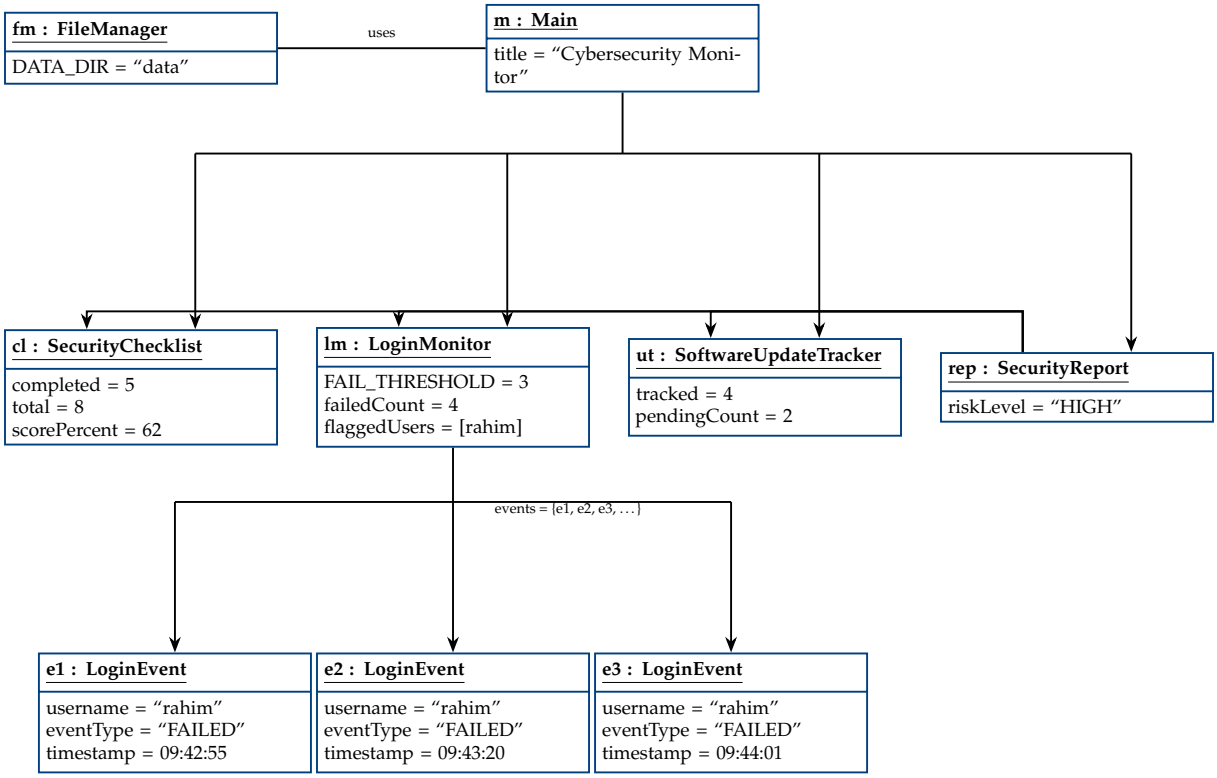


Figure 8.9: Object diagram – snapshot of the system with sample data

9 Implementation Plan and User Interface

9.1 Project Structure

The source code follows the folder structure required for the project:

```
CybersecurityMonitor/
|
|-- Main.java
|-- SecurityChecklist.java
|-- LoginMonitor.java
|-- SoftwareUpdateTracker.java
|-- SecurityReport.java
|-- FileManager.java
|
`-- data/
    |-- security.txt
    |-- login_events.txt
    `-- software_updates.txt
```

9.2 Class Responsibilities

Table 9.1: Classes used in the implementation (all appear in the class diagram)

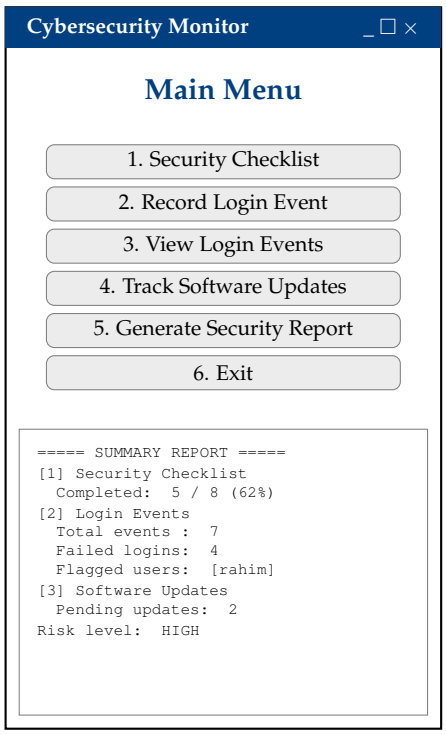
Class	Responsibility
`Main`	Entry point; builds the Swing window with six menu buttons and a console area; dispatches the selected option (decision "Select an Option").
`SecurityChecklist`	Holds the eight recommended security practices, shows the checkbox dialog, saves status, computes the score.
`LoginMonitor`	Records login events, reads them back, counts failures, and decides "Repeated Failed Logins?" (threshold = 3).
`SoftwareUpdateTracker`	Saves and reads software update status; returns the latest status per program and the number of pending updates.
`SecurityReport`	Reads all saved records through the other classes and builds the summary text including the risk level.
`FileManager`	Creates the `data/` folder and files; generic `appendLine`, `writeAll` and `readAll` operations.
`LoginMonitor.LoginEvent`	Small nested data class representing one login record.

9.3 Report Risk Rule

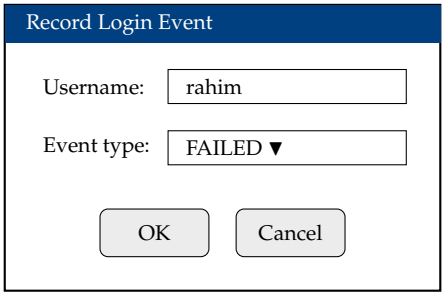
The risk level in `SecurityReport.riskLevel()` is computed from a simple score: checklist below 50% adds 2 points (below 80% adds 1); any flagged user adds 2; any pending software update adds 1. A total of 4 or more is **HIGH**, 2–3 is **MEDIUM**, otherwise **LOW**.

9.4 User Interface Design

The interface is a simple Swing window (`JFrame`) with a title, six buttons that correspond exactly to the menu options of the work flow, and a read-only console area that logs actions and shows the last report. Each task opens a small dialog (`JOptionPane`) and then returns to the menu. Figure 9.1 shows the interface design.



(a) Main window with report in the console



(b) Record Login Event dialog



(c) Security alert dialog

Figure 9.1: Interface design (the implemented Swing windows follow this layout)

9.5 Build and Run Instructions

```
cd CybersecurityMonitor
javac *.java          # compile all classes
java Main             # start the application
```

The `data/` folder is created automatically on first run if it does not exist.

9.6 Source Code of the Initial Version

The following listings contain the initial working version (v0.1 skeleton extended with all six classes) that accompanies this proposal. It compiles with JDK 17 and was smoke-tested with the sample data of Appendix A.

```
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.ArrayList;
import java.util.List;

/**
 * FileManager - the only class that touches the file system.
 * It stores and retrieves plain-text records inside the data/ folder.
 */
public class FileManager {
    public static final String DATA_DIR = "data";
    public static final String SECURITY_FILE = "security.txt";
    public static final String LOGIN_FILE    = "login_events.txt";
    public static final String UPDATE_FILE   = "software_updates.txt";

    /** Creates the data folder and the three text files if missing. */
    public void initialize() {
        try {
            Files.createDirectories(Paths.get(DATA_DIR));
            for (String f : new String[]{SECURITY_FILE, LOGIN_FILE, UPDATE_FILE}) {
                Path p = Paths.get(DATA_DIR, f);
                if (Files.notExists(p)) Files.createFile(p);
            }
        } catch (IOException e) {
            throw new UncheckedIOException("Cannot initialise data folder", e);
        }
    }

    /** Appends one line to a data file. */
    public void appendLine(String fileName, String line) {
        try (BufferedWriter w = Files.newBufferedWriter(Paths.get(DATA_DIR, fileName),
                StandardCharsets.UTF_8, StandardOpenOption.CREATE, StandardOpenOption.APPEND
                    )) {
            w.write(line);
            w.newLine();
        } catch (IOException e) {
            throw new UncheckedIOException("Write failed: " + fileName, e);
        }
    }

    /** Replaces the full content of a data file. */
    public void writeAll(String fileName, List<String> lines) {
        try {
            Files.write(Paths.get(DATA_DIR, fileName), lines, StandardCharsets.UTF_8);
        } catch (IOException e) {
            throw new UncheckedIOException("Write failed: " + fileName, e);
        }
    }

    /** Reads all non-empty lines from a data file. */
```

```
    public List<String> readAll(String fileName) {
        List<String> result = new ArrayList<>();
        Path p = Paths.get(DATA_DIR, fileName);
        if (Files.notExists(p)) return result;
        try {
            for (String s : Files.readAllLines(p, StandardCharsets.UTF_8)) {
                if (!s.trim().isEmpty()) result.add(s);
            }
        } catch (IOException e) {
            throw new UncheckedIOException("Read failed: " + fileName, e);
        }
        return result;
    }
}
```

Listing 9.1: `FileManager.java`

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;

/**
 * LoginMonitor - records login events (SUCCESS / FAILED) and detects
 * repeated failed logins for the same user.
 * File format (data/login_events.txt):  timestamp|username|eventType
 */
public class LoginMonitor {
    public static final int FAIL_THRESHOLD = 3;
    private static final DateTimeFormatter FMT = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:
        mm:ss");

    /** One saved login event. */
    public static class LoginEvent {
        private final String timestamp, username, eventType;
        public LoginEvent(String timestamp, String username, String eventType) {
            this.timestamp = timestamp; this.username = username; this.eventType = eventType
                ;
        }
        public String getTimestamp() { return timestamp; }
        public String getUsername()  { return username; }
        public String getEventType() { return eventType; }
        public boolean isFailed()    { return "FAILED".equals(eventType); }
        @Override public String toString() { return timestamp + "|" + username + "|" +
            eventType; }
    }

    private final FileManager fileManager;

    public LoginMonitor(FileManager fileManager) { this.fileManager = fileManager; }

    /** Save Login Event. */
    public LoginEvent record(String username, String eventType) {
        LoginEvent ev = new LoginEvent(LocalDateTime.now().format(FMT), username.trim(),
            eventType);
        fileManager.appendLine(FileManager.LOGIN_FILE, ev.toString());
        return ev;
    }

    /** Read Saved Login Records. */
    public List<LoginEvent> getEvents() {
        List<LoginEvent> list = new ArrayList<>();
        for (String line : fileManager.readAll(FileManager.LOGIN_FILE)) {
            String[] p = line.split("\\|");
            if (p.length == 3) list.add(new LoginEvent(p[0], p[1], p[2]));
        }
        return list;
    }

    /** Decision "Repeated Failed Logins?" - true when the latest N events of the user are
        all FAILED. */
    public boolean hasRepeatedFailures(String username) {
        int streak = 0;
        List<LoginEvent> all = getEvents();
```

```
        for (int i = all.size() - 1; i >= 0; i--) {
            LoginEvent e = all.get(i);
            if (!e.getUsername().equalsIgnoreCase(username)) continue;
            if (e.isFailed()) streak++; else break;
        }
        return streak >= FAIL_THRESHOLD;
    }

    public int countFailed() {
        int n = 0;
        for (LoginEvent e : getEvents()) if (e.isFailed()) n++;
        return n;
    }

    /** Usernames whose current failure streak reached the threshold. */
    public Set<String> getFlaggedUsers() {
        Set<String> users = new LinkedHashSet<>(), flagged = new LinkedHashSet<>();
        for (LoginEvent e : getEvents()) users.add(e.getUsername().toLowerCase());
        for (String u : users) if (hasRepeatedFailures(u)) flagged.add(u);
        return flagged;
    }
}
```

Listing 9.2: `LoginMonitor.java`

```
import javax.swing.*;
import java.awt.*;
import java.util.*;
import java.util.List;

/**
 * SecurityChecklist - manages the list of recommended security practices
 * and stores their completion status in data/security.txt.
 */
public class SecurityChecklist {
    public static final String[] ITEMS = {
        "Use a strong, unique password for every account",
        "Enable two-factor authentication (2FA)",
        "Keep the operating system up to date",
        "Install and update antivirus software",
        "Enable the firewall",
        "Back up important files regularly",
        "Avoid clicking unknown links and attachments",
        "Use encrypted (HTTPS/VPN) connections on public Wi-Fi"
    };

    private final FileManager fileManager;
    private final Map<String, Boolean> status = new LinkedHashMap<>();

    public SecurityChecklist(FileManager fileManager) {
        this.fileManager = fileManager;
        load();
    }

    /** Loads saved status; unsaved items default to "pending". */
    public final void load() {
        status.clear();
        for (String item : ITEMS) status.put(item, false);
        for (String line : fileManager.readAll(FileManager.SECURITY_FILE)) {
            String[] p = line.split("\\|");
            if (p.length == 2 && status.containsKey(p[0])) status.put(p[0], p[1].equals("
                DONE"));
        }
    }

    /** Saves the status of every checklist item (Save Security Status). */
    public void save() {
        List<String> lines = new ArrayList<>();
        for (Map.Entry<String, Boolean> e : status.entrySet())
            lines.add(e.getKey() + "|" + (e.getValue() ? "DONE" : "PENDING"));
        fileManager.writeAll(FileManager.SECURITY_FILE, lines);
    }

```

```
    public int getCompleted() {
        int n = 0;
        for (boolean b : status.values()) if (b) n++;
        return n;
    }

    public int getTotal() { return status.size(); }

    public int getScorePercent() { return getTotal() == 0 ? 0 : getCompleted() * 100 /
        getTotal(); }

    /** Shows the checklist dialog and saves the result when OK is pressed. */
    public void showDialog(Component parent) {
        load();
        JPanel panel = new JPanel(new GridLayout(0, 1, 2, 4));
        List<JCheckBox> boxes = new ArrayList<>();
        for (Map.Entry<String, Boolean> e : status.entrySet()) {
            JCheckBox cb = new JCheckBox(e.getKey(), e.getValue());
            boxes.add(cb);
            panel.add(cb);
        }
        int r = JOptionPane.showConfirmDialog(parent, panel, "Security Checklist",
                JOptionPane.OK_CANCEL_OPTION, JOptionPane.PLAIN_MESSAGE);
        if (r == JOptionPane.OK_OPTION) {
            int i = 0;
            for (String key : new ArrayList<>(status.keySet())) status.put(key, boxes.get(i
                ++).isSelected());
            save();
            JOptionPane.showMessageDialog(parent,
                "Saved. Security score: " + getScorePercent() + "% (" + getCompleted() + "/"
                     + getTotal() + ")");
        }
    }
}
```

Listing 9.3: `SecurityChecklist.java`

```
import java.time.LocalDate;
import java.util.*;

/**
 * SoftwareUpdateTracker - keeps a record of installed software and whether
 * its update is UP-TO-DATE or PENDING.
 * File format (data/software_updates.txt):  software|version|status|date
 */
public class SoftwareUpdateTracker {
    public static final String UP_TO_DATE = "UP-TO-DATE";
    public static final String PENDING = "PENDING";

    private final FileManager fileManager;

    public SoftwareUpdateTracker(FileManager fileManager) { this.fileManager = fileManager;
        }

    /** Save Update Status. */
    public void saveUpdate(String software, String version, String status) {
        fileManager.appendLine(FileManager.UPDATE_FILE,
            software.trim() + "|" + version.trim() + "|" + status + "|" + LocalDate.now());
    }

    public List<String[]> getAll() {
        List<String[]> rows = new ArrayList<>();
        for (String line : fileManager.readAll(FileManager.UPDATE_FILE)) {
            String[] p = line.split("\\|");
            if (p.length == 4) rows.add(p);
        }
        return rows;
    }

    /** Latest status per software name (later records override earlier ones). */
    public Map<String, String> latestStatus() {
        Map<String, String> m = new LinkedHashMap<>();
        for (String[] r : getAll()) m.put(r[0], r[2]);
```

```
        return m;
    }

    public int countPending() {
        int n = 0;
        for (String s : latestStatus().values()) if (PENDING.equals(s)) n++;
        return n;
    }
}
```

Listing 9.4: `SoftwareUpdateTracker.java`

```
import java.time.LocalDateTime;
import java.util.Map;

/**
 * SecurityReport - reads all saved records and builds a summary report.
 */
public class SecurityReport {
    private final SecurityChecklist checklist;
    private final LoginMonitor loginMonitor;
    private final SoftwareUpdateTracker updateTracker;

    public SecurityReport(SecurityChecklist c, LoginMonitor l, SoftwareUpdateTracker u) {
        this.checklist = c; this.loginMonitor = l; this.updateTracker = u;
    }

    /** Read All Saved Records -> Display Summary Report. */
    public String generate() {
        checklist.load();
        StringBuilder sb = new StringBuilder();
        sb.append("===== CYBERSECURITY SUMMARY REPORT =====\n");
        sb.append("Generated: ").append(LocalDateTime.now().withNano(0)).append("\n\n");

        sb.append("[1] Security Checklist\n");
        sb.append("    Completed : ").append(checklist.getCompleted()).append(" / ")
          .append(checklist.getTotal()).append("  (").append(checklist.getScorePercent()).
              append("%)\n\n");

        sb.append("[2] Login Events\n");
        sb.append("    Total events  : ").append(loginMonitor.getEvents().size()).append("\n
            ");
        sb.append("    Failed logins : ").append(loginMonitor.countFailed()).append("\n");
        sb.append("    Flagged users : ").append(loginMonitor.getFlaggedUsers().isEmpty()
                ? "none" : loginMonitor.getFlaggedUsers().toString()).append("\n\n");

        sb.append("[3] Software Updates\n");
        Map<String, String> st = updateTracker.latestStatus();
        if (st.isEmpty()) sb.append("    No software tracked yet.\n");
        for (Map.Entry<String, String> e : st.entrySet())
            sb.append("    - ").append(e.getKey()).append(" : ").append(e.getValue()).append
                ("\n");
        sb.append("    Pending updates: ").append(updateTracker.countPending()).append("\n\n
            ");

        sb.append("Risk level: ").append(riskLevel()).append("\n");
        return sb.toString();
    }

    public String riskLevel() {
        int risk = 0;
        if (checklist.getScorePercent() < 50) risk += 2; else if (checklist.getScorePercent
            () < 80) risk += 1;
        if (!loginMonitor.getFlaggedUsers().isEmpty()) risk += 2;
        if (updateTracker.countPending() > 0) risk += 1;
        return risk >= 4 ? "HIGH" : risk >= 2 ? "MEDIUM" : "LOW";
    }
}
```

Listing 9.5: `SecurityReport.java`

```
import javax.swing.*;
```

```
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.List;

/**
 * Main - launches the application and shows the Main Menu (options 1-6).
 */
public class Main extends JFrame {
    private final FileManager fileManager = new FileManager();
    private final SecurityChecklist checklist;
    private final LoginMonitor loginMonitor;
    private final SoftwareUpdateTracker updateTracker;
    private final SecurityReport report;
    private final JTextArea console = new JTextArea(10, 40);

    public Main() {
        super("Cybersecurity Monitor");
        fileManager.initialize();
        checklist     = new SecurityChecklist(fileManager);
        loginMonitor  = new LoginMonitor(fileManager);
        updateTracker = new SoftwareUpdateTracker(fileManager);
        report        = new SecurityReport(checklist, loginMonitor, updateTracker);
        buildMenu();
    }

    private void buildMenu() {
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new java.awt.event.WindowAdapter() {
            @Override public void windowClosing(java.awt.event.WindowEvent e) {
                exitApplication(); }
        });
        setLayout(new BorderLayout(8, 8));

        JLabel title = new JLabel("Cybersecurity Monitor - Main Menu", SwingConstants.CENTER
            );
        title.setFont(title.getFont().deriveFont(Font.BOLD, 18f));
        add(title, BorderLayout.NORTH);

        JPanel menu = new JPanel(new GridLayout(0, 1, 6, 6));
        menu.setBorder(BorderFactory.createEmptyBorder(8, 16, 8, 16));
        String[] labels = {"1. Security Checklist", "2. Record Login Event", "3. View Login
            Events",
                           "4. Track Software Updates", "5. Generate Security Report", "6.
                               Exit"};
        for (int i = 0; i < labels.length; i++) {
            final int option = i + 1;
            JButton b = new JButton(labels[i]);
            b.addActionListener(e -> handleOption(option));
            menu.add(b);
        }
        add(menu, BorderLayout.CENTER);

        console.setEditable(false);
        console.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
        console.setText("Welcome. Choose an option from the menu.\n");
        add(new JScrollPane(console), BorderLayout.SOUTH);

        pack();
        setLocationRelativeTo(null);
    }

    /** Decision node "Select an Option". */
    private void handleOption(int option) {
        switch (option) {
            case 1: checklist.showDialog(this); log("Checklist saved. Score: " + checklist.
                getScorePercent() + "%"); break;
            case 2: recordLogin(); break;
            case 3: viewLoginEvents(); break;
            case 4: trackUpdates(); break;
            case 5: showReport(); break;
            case 6: exitApplication(); break;
        }
    }
```

```
70
71      private void recordLogin() {
72          JTextField user = new JTextField(15);
73          JComboBox<String> type = new JComboBox<>(new String[]{"SUCCESS", "FAILED"});
74          JPanel p = new JPanel(new GridLayout(0, 2, 4, 4));
75          p.add(new JLabel("Username:")); p.add(user);
76          p.add(new JLabel("Event type:")); p.add(type);
77          if (JOptionPane.showConfirmDialog(this, p, "Record Login Event",
78                  JOptionPane.OK_CANCEL_OPTION) != JOptionPane.OK_OPTION) return;
79          if (user.getText().trim().isEmpty()) {
80              JOptionPane.showMessageDialog(this, "Username cannot be empty.", "Input error",
                    JOptionPane.ERROR_MESSAGE);
81              return;
82          }
83          loginMonitor.record(user.getText(), (String) type.getSelectedItem());
84          log("Saved login event for '" + user.getText().trim() + "' (" + type.getSelectedItem
                () + ")");
85          if (loginMonitor.hasRepeatedFailures(user.getText().trim())) {          // Repeated
                Failed Logins?
86              JOptionPane.showMessageDialog(this, "SECURITY ALERT: " + LoginMonitor.
                    FAIL_THRESHOLD
87                  + " or more failed logins for '" + user.getText().trim() + "'!",
88                  "Security Alert", JOptionPane.WARNING_MESSAGE);                  // Display
                        Security Alert
89              log("ALERT: repeated failed logins for " + user.getText().trim());
90          }                                                                        // else:
                Continue
91      }
92
93      private void viewLoginEvents() {
94          DefaultTableModel model = new DefaultTableModel(new String[]{"Timestamp", "Username"
                , "Event"}, 0);
95          for (LoginMonitor.LoginEvent e : loginMonitor.getEvents())
96              model.addRow(new Object[]{e.getTimestamp(), e.getUsername(), e.getEventType()});
97          JTable table = new JTable(model);
98          JScrollPane sp = new JScrollPane(table);
99          sp.setPreferredSize(new Dimension(480, 220));
100         JOptionPane.showMessageDialog(this, sp, "Login Events (" + model.getRowCount() + ")"
                , JOptionPane.PLAIN_MESSAGE);
101     }
102
103     private void trackUpdates() {
104         JTextField name = new JTextField(15), ver = new JTextField(15);
105         JComboBox<String> st = new JComboBox<>(new String[]{SoftwareUpdateTracker.UP_TO_DATE
                , SoftwareUpdateTracker.PENDING});
106         JPanel p = new JPanel(new GridLayout(0, 2, 4, 4));
107         p.add(new JLabel("Software:")); p.add(name);
108         p.add(new JLabel("Version:"));  p.add(ver);
109         p.add(new JLabel("Status:"));   p.add(st);
110         if (JOptionPane.showConfirmDialog(this, p, "Track Software Updates",
111                 JOptionPane.OK_CANCEL_OPTION) != JOptionPane.OK_OPTION) return;
112         if (name.getText().trim().isEmpty()) return;
113         updateTracker.saveUpdate(name.getText(), ver.getText(), (String) st.getSelectedItem
                ());
114         log("Saved update status: " + name.getText().trim() + " -> " + st.getSelectedItem())
                ;
115     }
116
117     private void showReport() {
118         String text = report.generate();
119         console.setText(text);
120         JTextArea area = new JTextArea(text, 18, 46);
121         area.setEditable(false);
122         area.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
123         JOptionPane.showMessageDialog(this, new JScrollPane(area), "Security Report",
                JOptionPane.PLAIN_MESSAGE);
124     }
125
126     private void exitApplication() {
127         if (JOptionPane.showConfirmDialog(this, "Exit the application?", "Exit",
128                 JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION) {
129             dispose();
130             System.exit(0);                                                      // End
```

```
131             }
132         }
133
134         private void log(String msg) { console.append(msg + "\n"); }
135
136         public static void main(String[] args) {                                    // Start ->
                Launch Java Application
137             SwingUtilities.invokeLater(() -> new Main().setVisible(true));          // Display
                    Main Menu
138         }
139     }
```

Listing 9.6: `Main.java`

10 Testing Plan

Testing is performed at the end of every sprint (unit and integration level) and as a complete system test in weeks 13–14. Table 10.1 lists the planned test cases.

Table 10.1: Planned test cases

ID	Test	Input / action	Expected result
TC-01	Application start	Run `java Main`	Main menu with 6 buttons; `data/` files exist
TC-02	Record successful login	user = alif, SUCCESS	Line appended to `login_events.txt`; no alert
TC-03	Empty user name	user = (blank)	Error message; nothing saved
TC-04	Repeated failures	3 × FAILED for rahim	Security alert shown after third event
TC-05	Failures interrupted	FAILED, FAILED, SUCCESS	No alert
TC-06	View events	Option 3	Table lists all saved events
TC-07	Checklist save/reload	Tick 5 items, OK, reopen	Same 5 ticked; score 62%
TC-08	Track update	Chrome, 130, PENDING	Record saved; pending count increases
TC-09	Latest status wins	Save Chrome UP-TO-DATE after PENDING	Report shows UP-TO-DATE
TC-10	Generate report	Option 5	Report shows score, failures, flagged users, pending updates, risk
TC-11	Corrupt data line	Add random text to `login_events.txt`	Line ignored; application does not crash
TC-12	Exit	Option 6 / close window, confirm	Application terminates; Cancel keeps it open

Pass criteria. All high-priority requirements (FR-1 to FR-4, FR-6, FR-7, FR-9) must pass; no crash is allowed on invalid input.

11 Expected Outcomes, Limitations and Future Work

11.1 Expected Outcomes

- A tested desktop application with six menu options and at least six cooperating classes.
- Persistent, human-readable records of checklist status, login events and software updates.
- An automatic alert after three consecutive failed logins and a risk rating in the report.
- Complete project documentation: proposal, UML models, test report and user manual.

11.2 Limitations

Login events are entered manually, the update status is user-declared, data files are not encrypted and the application is single-user. These limits are deliberate to keep the project achievable in one semester.

11.3 Future Work

- Automatic reading of operating-system login logs.
- Export of the report to PDF and e-mail notification of alerts.
- Migration from text files to SQLite using the ERD of Chapter 8.
- Password-strength checker, charts of events over time and a role-based login screen.

11.4 Conclusion

CyberSecurity Monitor addresses a real and common need – seeing the basic security state of a computer at a glance – using a small, well-structured Java application. The incremental Scrum process, detailed schedule with Gantt chart, UML models and working code skeleton presented here show that the project is clearly defined, technically feasible and manageable within 16 weeks.

Bibliography

[1] I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2016.

[2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed., McGraw-Hill, 2020.

[3] K. Schwaber and J. Sutherland, *The Scrum Guide*, 2020. https://scrumguides.org

[4] G. Booch, J. Rumbaugh and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd ed., Addison-Wesley, 2005.

[5] P. Chen, "The Entity-Relationship Model – Toward a Unified View of Data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.

[6] T. DeMarco, *Structured Analysis and System Specification*, Yourdon Press, 1979.

[7] National Institute of Standards and Technology, *The NIST Cybersecurity Framework (CSF) 2.0*, 2024.

[8] Center for Internet Security, *CIS Critical Security Controls, Version 8*, 2021.

[9] OWASP Foundation, *Authentication Cheat Sheet*. https://cheatsheetseries.owasp.org

[10] Oracle, *The Java Tutorials – Creating a GUI with Swing*. https://docs.oracle.com/javase/tutorial/uiswing/

[11] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed., 2021.

A Sample Data Files

login_events.txt

```
2026-11-02 09:10:12|alif|SUCCESS
2026-11-02 09:42:55|rahim|FAILED
2026-11-02 09:43:20|rahim|FAILED
2026-11-02 09:44:01|rahim|FAILED
2026-11-02 10:05:31|karim|SUCCESS
2026-11-02 11:20:09|alif|FAILED
2026-11-02 11:20:40|alif|SUCCESS
```

software_updates.txt

```
Google Chrome|130.0|UP-TO-DATE|2026-11-02
Windows 11|23H2|PENDING|2026-11-02
Java JDK|17.0.9|UP-TO-DATE|2026-11-02
VLC Player|3.0.18|PENDING|2026-11-02
```

security.txt

```
Use a strong, unique password for every account|DONE
Enable two-factor authentication (2FA)|DONE
Keep the operating system up to date|PENDING
Install and update antivirus software|DONE
Enable the firewall|DONE
Back up important files regularly|PENDING
Avoid clicking unknown links and attachments|DONE
Use encrypted (HTTPS/VPN) connections on public Wi-Fi|PENDING
```